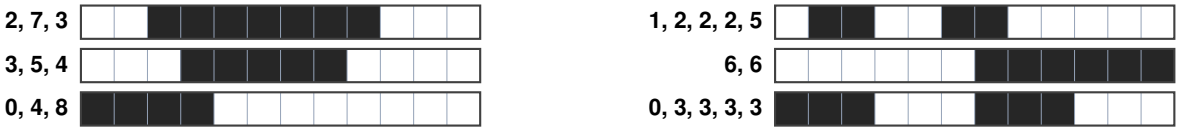
INSTRUCTOR COPY Cryptography All-in-One: answers

Page numbers are the pupil booklet's. Pictures are named here only: pupils are never told the names on paper.

Page 2 Warm-up rows

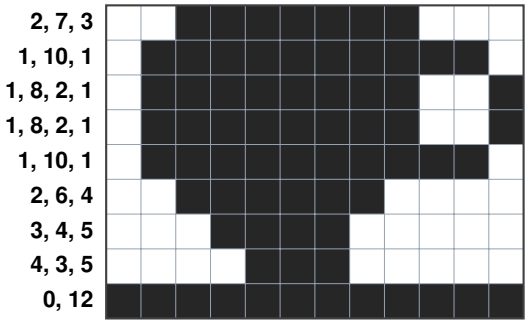


Squash it: `AAAABBBCCCCD` → `4A 3B 4C 1D` `WWWWWWBBW` → `6W 2B 1W` `XXXXXXXXXX` → `10X`

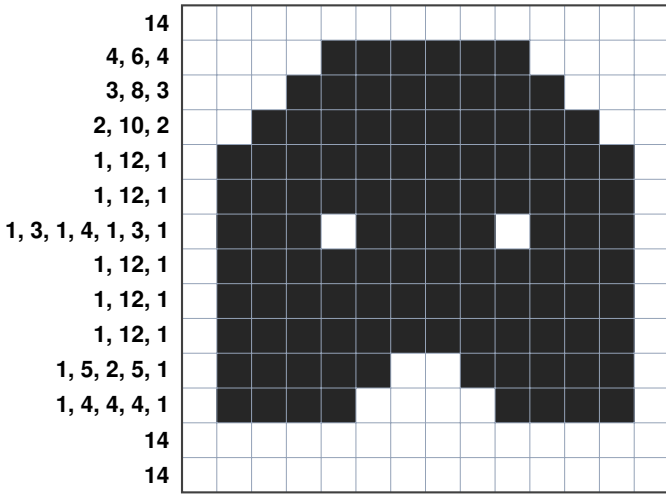
Stretch it: `4C 2A 5B` → `CCCCAABBBBB` `3X 1Y 2Z` → `XXXYZZ` `1A 1B 1A 1B 1A` → `ABABA`

Pages 3 to 5 By-hand pictures

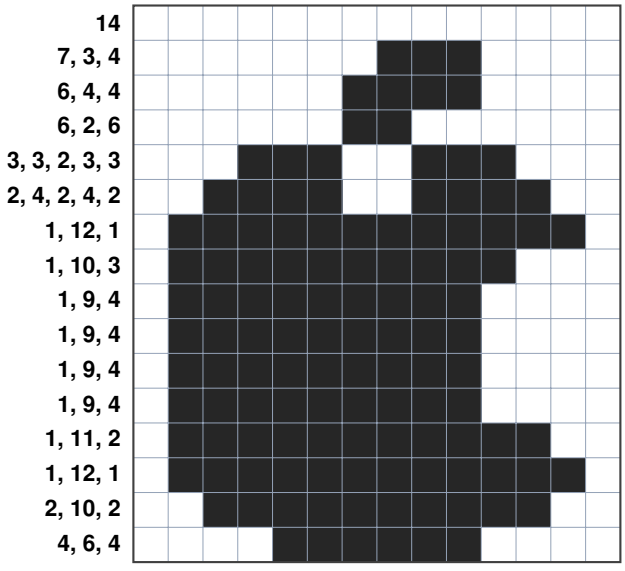
Page 3: Mystery 1 = mug



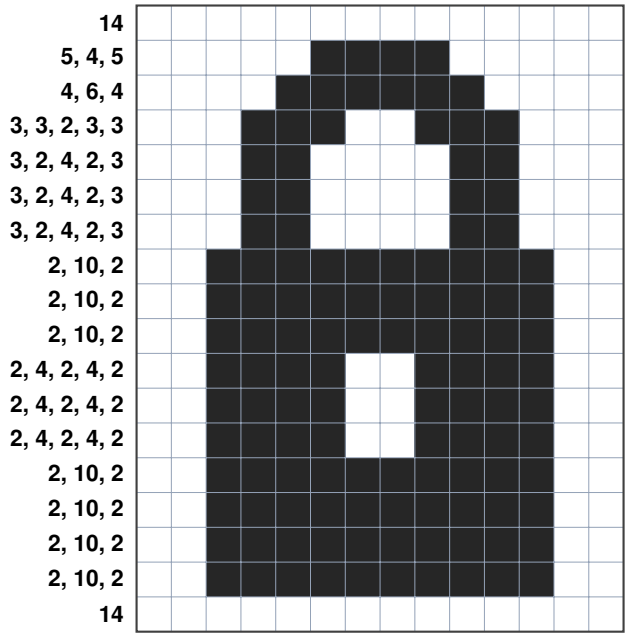
Page 5: Mystery 4 = ghost



Page 4: Mystery 2 = Apple logo



Page 4: Mystery 3 = padlock



Page 5 pattern questions (checked by script): same clue: rows 1, 13, 14; rows 5, 6, 8, 9, 10. Widest row (most coloured pixels): row 5, 6, 8, 9, 10. Left and right: the left and right sides mirror each other (every row reads the same backwards).

Page 7 Be the computer

`decode_row([2, 7, 3])`

step	no.	col.	added	row so far
1	2	0	0, 0	[0, 0]
2	7	1	1 ×7	[0, 0, 1, 1, 1, 1, 1, 1, 1]
3	3	0	0 ×3	[0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0]

`decode_row([0, 4, 2])`

step	no.	col.	added	row so far
1	0	0	nothing	[]
2	4	1	1 ×4	[1, 1, 1, 1]
3	2	0	0 ×2	[1, 1, 1, 1, 0, 0]

Different about the first step: `range(0)` runs zero times, so nothing is added, but the colour still swaps. The row starts with 1.

`colour = 1 - colour` **:** 0 becomes 1 $(1 - 0)$ and 1 becomes 0 $(1 - 1)$. It flips the colour.

Page 8 Plan before you type: model answers

`decode_row`

1. Start with an empty row and colour 0.
2. For each number in the clue:
3. add that many copies of colour to the row.
4. Swap colour (0 to 1, 1 to 0).
5. Return the row.

`decode_picture`

1. Start with an empty picture.
2. For each clue: decode it into a row.
3. Add the row to the picture.
4. Return the picture.

Pages 9 to 10 Completed code (same as `code/solutions/rle_solution.py`)

```
def decode_row(clue):
    row = []
    colour = 0
    for number in clue:
        for i in range(number):
            row.append(colour)
        if colour == 0:
            colour = 1
        else:
            colour = 0
    return row
```

```
def decode_picture(clues):
    picture = []
    for clue in clues:
        row = decode_row(clue)
        picture.append(row)
    return picture
```

```
def encode_row(row):
    clue = []
    colour = 0
    count = 0
    for pixel in row:
        if pixel == colour:
            count += 1
        else:
            clue.append(count)
            colour = pixel
            count = 1
    clue.append(count)
    return clue
```

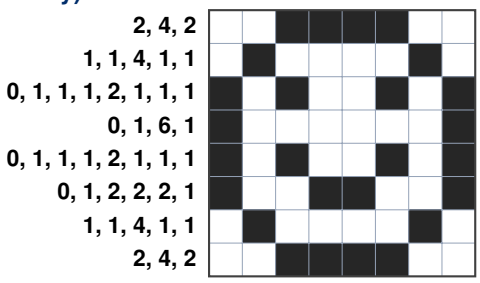
Gaps in the hints: page 9 hint 3: `range(number)` , `row.append(colour)` , `colour = 1 - colour` . Page 10 hint 3: `clues` , `decode_row` , `append` . Test output: `[0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0]` . Stretch (page 12): `encode_row` of that row gives `[2, 7, 3]` .

Page 11 Big pictures and timings

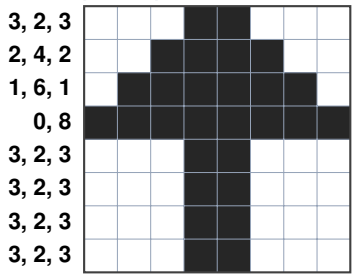
Row counts printed in the table are recomputed from `pictures.py` : mug 9, `DINOSAUR` 47, `MARIO` 32, `UK_IRELAND` 46. The estimate on the page uses 46/9 ≈ 5 times as many rows. Times vary: expect the computer to take under a second for every picture. The code does not need to change for the big pictures (that is the point). The by-hand grids for these three pictures are in `other-materials/pdf` (09, 10, 11) if a pupil wants to race the computer.

Page 12 Stretch: encode it

Picture A (smiley)



Picture B (arrow)



`encode_row` solution is lines 34 to 46 of `rle_solution.py` (above, right column). Common slip: forgetting the final `clue.append(count)`.